Repaso del día 6

16 preguntas sobre control de acceso, Vault, mappings, reentrancy, coste real, EVM vs Fabric y diseño de Smart Contracts

Las 16 preguntas — Repaso día 6

1. Si todas las transacciones de un contrato son públicas, ¿por qué un contrato necesita saber QUIÉN lo está llamando?
2. ¿Qué diferencia hay entre proteger una función con "solo el dueño puede ejecutarla" y con "una lista de autorizados puede ejecutarla"?
3. ¿Qué pasa si la wallet que es "owner" de un contrato se pierde, se filtra o muere su dueño? ¿Hay forma de recuperar el control?
4. ¿Por qué hay que marcar una función como "payable" para que pueda recibir ETH? ¿De qué nos protege?
5. ¿Cómo es posible que un contrato como un token lleve la cuenta del saldo de millones de wallets sin que su coste de gas crezca con cada nuevo usuario?
6. Un contrato Vault usa block.timestamp para saber si han pasado 60 segundos. ¿Se puede confiar en el reloj del bloque? ¿Quién lo decide?
7. ¿Qué diferencia hay entre que un contrato "emita un evento" y que "guarde un dato" en una variable? Si ambos quedan en blockchain, ¿no es lo mismo?
8. ¿Por qué importa tanto el orden de las operaciones dentro de una función? ¿Qué cambia si primero envías dinero y luego actualizas el saldo, o al revés?

9. ¿Qué es un ataque de reentrancy y por qué es tan peligroso en blockchain pero no en un servidor web normal?

10. ¿Por qué desplegar exactamente el mismo contrato en Ethereum mainnet puede costar 50 dólares pero en Polygon menos de 1 céntimo? ¿No es la misma operación?

11. Si una testnet como Sepolia funciona exactamente igual que mainnet, ¿por qué su ETH no tiene valor? ¿Qué la hace "de prueba"?

12. Si despliegas un contrato y luego mil personas lo usan, ¿pagas tú el gas de esas mil interacciones o lo pagan ellas?

13. Si Ethereum y Hyperledger Fabric persiguen lo mismo (ejecutar lógica de negocio en una blockchain), ¿por qué Fabric NO usa Solidity ni cobra gas?

14. Un contrato desplegado lo puede invocar cualquiera del mundo. ¿Es eso una ventaja o un riesgo? ¿Cómo te proteges?

15. ¿Por qué MetaMask pide firma para cada interacción con un contrato y no funciona como un "login" donde inicias sesión una vez? ¿Qué riesgo aparece si te acostumbras a firmar sin leer?

16. Si quisieras guardar una contraseña, un DNI o un dato personal en un Smart Contract, ¿podrías? ¿Por qué es una mala idea?

PREGUNTA 1 / 16

Identidad en un contrato

Pregunta

Si todas las transacciones de un contrato son públicas, ¿por qué un contrato necesita saber QUIÉN lo está llamando?

Respuesta razonada

Aunque todo el mundo pueda ver las transacciones, no todo el mundo debería poder hacer las mismas acciones. Quien despliega un Vault no quiere que un desconocido pueda retirar sus fondos, y quien crea una whitelist no quiere que cualquiera la modifique. El contrato lee msg.sender para distinguir 'esto lo está pidiendo el dueño' de 'esto lo está pidiendo otro'. Sin esa distinción, todas las funciones serían públicas para todos — y eso convierte el contrato en un cajero al que cualquiera mete la mano.

PREGUNTA 2 / 16

Owner vs Whitelist

Pregunta

¿Qué diferencia hay entre proteger una función con "solo el dueño puede ejecutarla" y con "una lista de autorizados puede ejecutarla"?

Respuesta razonada

Son dos modelos de control. Solo el dueño es un acceso muy concentrado: una sola wallet manda. Es simple pero peligroso (si pierdes esa wallet, pierdes el control). La whitelist es un acceso distribuido: el dueño decide a quién meter en la lista, pero luego varios pueden actuar. Es más flexible para escenarios reales (operadores, equipo, sucursales), pero introduce un segundo nivel de gobernanza: quién mantiene la lista. Casi todos los contratos serios combinan ambos: un owner que administra una whitelist.

PREGUNTA 3 / 16

Perder la llave maestra

Pregunta

¿Qué pasa si la wallet que es "owner" de un contrato se pierde, se filtra o muere su dueño? ¿Hay forma de recuperar el control?

Respuesta razonada

No, salvo que el contrato lo haya previsto desde el principio. Si simplemente comprueba msg.sender == owner, la dirección antigua queda atrapada para siempre y el contrato pierde su administración. Por eso los contratos serios incluyen transferOwnership (cambiar de dueño), 2-step transfer (el nuevo dueño debe aceptar antes de tomar control) o gobernanza por multisig/DAO. Es la versión cripto del "no pongas todos los huevos en una cesta": una sola clave manejando todo es una bomba de relojería.

PREGUNTA 4 / 16

Funciones payable

Pregunta

¿Por qué hay que marcar una función como "payable" para que pueda recibir ETH? ¿De qué nos protege?

Respuesta razonada

Es una protección por defecto. Si todas las funciones aceptaran ETH automáticamente, sería muy fácil enviar dinero a una función que no sabe qué hacer con él, y quedar atrapado. Marcar payable es decir explícitamente: "sí, esta función está preparada para recibir y manejar ETH". Si alguien intenta enviar ETH a una función no payable, la transacción se revierte y se devuelve el dinero. Es el sistema diciendo: "si no me has dicho que aceptas pagos, asumo que ha sido un error".

PREGUNTA 5 / 16

Mappings

Pregunta

¿Cómo es posible que un contrato como un token lleve la cuenta del saldo de millones de wallets sin que su coste de gas crezca con cada nuevo usuario?

Respuesta razonada

Porque un mapping no almacena nada hasta que se usa. Conceptualmente es una tabla infinita donde TODAS las direcciones tienen valor 0 por defecto. Solo cuando alguien escribe en una entrada concreta ("saldo de Ana = 100") se paga el coste de almacenamiento de ESA entrada. Para todas las demás direcciones (los millones que aún no han recibido nada) no se paga nada. El mapping no "escala" porque no se inicializa: cobra solo por la información que realmente se ha grabado.

PREGUNTA 6 / 16

block.timestamp

Pregunta

Un contrato Vault usa block.timestamp para saber si han pasado 60 segundos. ¿Se puede confiar en el reloj del bloque? ¿Quién lo decide?

Respuesta razonada

El timestamp lo pone el validador que propone el bloque, no el contrato. Por eso tiene una pequeña tolerancia: un validador podría jugar con unos segundos arriba o abajo dentro de un margen estrecho (la red rechaza valores claramente fuera de rango). Para condiciones de minutos u horas es perfectamente fiable; para decidir un sorteo donde 1 segundo importa, es manipulable. Regla práctica: úsalo para vencimientos amplios (lock de 1 día, deadline de 1 hora), evítalo para precisión fina o para generar aleatoriedad.

PREGUNTA 7 / 16

Eventos vs storage

Pregunta

¿Qué diferencia hay entre que un contrato "emita un evento" y que "guarde un dato" en una variable? Si ambos quedan en blockchain, ¿no es lo mismo?

Respuesta razonada

No, son dos cosas distintas. Una variable de estado vive en el storage del contrato: cualquier función futura puede leerla y modificarla, y cuesta caro mantenerla. Un evento queda en los logs del bloque: es un mensaje histórico que el contrato NO puede volver a leer después. Sirve para que las aplicaciones externas (frontends, indexadores) reaccionen a cosas que pasaron. Resumen: si tu contrato lo necesita más tarde, guárdalo. Si solo lo necesitan las apps de fuera, emítelo. Lo segundo cuesta mucho menos.

PREGUNTA 8 / 16

Checks-Effects-Interactions

Pregunta

¿Por qué importa tanto el orden de las operaciones dentro de una función? ¿Qué cambia si primero envías dinero y luego actualizas el saldo, o al revés?

Respuesta razonada

Es la diferencia entre seguro y vulnerable. El patrón Checks-Effects-Interactions dice: primero comprueba (¿puede hacer esto?), luego modifica el estado (descontar saldo), y SOLO AL FINAL interactúa con el exterior (enviar el dinero). Si lo haces al revés (enviar primero, descontar después), el receptor puede volver a llamar a la función ANTES de que se descuente el saldo, y vaciar el contrato. Eso es reentrancy. The DAO perdió 60M USD por exactamente este motivo. Es una de esas reglas que parecen sutiles pero te ahorran fortunas.

PREGUNTA 9 / 16

Reentrancy

Pregunta

¿Qué es un ataque de reentrancy y por qué es tan peligroso en blockchain pero no en un servidor web normal?

Respuesta razonada

Es cuando un contrato externo, al recibir una llamada de tu contrato, te llama a TI de vuelta antes de que termines tu función original. Si tu función aún no había actualizado tu estado, el atacante repite la operación una y otra vez. En un servidor web normal, una llamada externa es una petición HTTP que termina antes de que continúes; en blockchain, la transacción es ATÓMICA y permite ese ida y vuelta dentro de la misma ejecución. Por eso el orden de las operaciones es crítico: cualquier interacción externa puede traer una sorpresa de vuelta.

PREGUNTA 10 / 16

Coste por red

Pregunta

¿Por qué desplegar exactamente el mismo contrato en Ethereum mainnet puede costar 50 dólares pero en Polygon menos de 1 céntimo? ¿No es la misma operación?

Respuesta razonada

Computacionalmente es la misma. Lo que cambia es el contexto económico: en mainnet hay millones de usuarios compitiendo por bloques limitados, y el precio del gas sube cuando hay demanda. Además, mainnet usa ETH (cotización alta) y Polygon usa MATIC (mucho más barato). El gas se paga en la moneda de cada red. Por eso un mismo despliegue puede ser irrelevante o prohibitivo según dónde lo hagas.

PREGUNTA 11 / 16

Testnets

Pregunta

Si una testnet como Sepolia funciona exactamente igual que mainnet, ¿por qué su ETH no tiene valor? ¿Qué la hace "de prueba"?

Respuesta razonada

Funciona técnicamente igual, pero el acuerdo de la comunidad es distinto. Mainnet tiene una economía real: ETH se compra y se vende con dinero real, hay validadores que invierten en hardware, exchanges que cotizan el precio. Sepolia es una infraestructura paralela que la comunidad usa solo para pruebas: el ETH se reparte gratis vía faucets, nadie lo compra, no hay precio. Si mañana todo el mundo decidiera "vamos a usar Sepolia como real", podría pasar — pero como no hay incentivos económicos detrás, no ocurre. La diferencia no es técnica, es social y económica.

PREGUNTA 12 / 16

¿Quién paga el gas?

Pregunta

Si despliegas un contrato y luego mil personas lo usan, ¿pagas tú el gas de esas mil interacciones o lo pagan ellas?

Respuesta razonada

Las pagan ellas. Tú pagas SOLO el despliegue (una vez) y cualquier transacción que TÚ firmes. Cuando otro usuario invoca una función de tu contrato, firma él la transacción, y por lo tanto él paga el gas. Es como abrir un quiosco: tú pagas montar el quiosco, pero cada cliente paga su café. Esto cambia con "meta-transactions" o "gasless transactions", donde un tercero (un relayer) puede pagar el gas en nombre del usuario para mejorar la UX — pero entonces alguien tiene que cubrir ese coste.

PREGUNTA 13 / 16

EVM vs Fabric

Pregunta

Si Ethereum y Hyperledger Fabric persiguen lo mismo (ejecutar lógica de negocio en una blockchain), ¿por qué Fabric NO usa Solidity ni cobra gas?

Respuesta razonada

Porque están diseñados para mundos distintos. Ethereum es una red pública sin permisos donde cualquiera puede entrar: necesita un mecanismo de gas para que nadie pueda saturarla gratis con bucles infinitos. Fabric es una red privada entre organizaciones que se conocen: ya hay confianza y reglas de gobernanza, así que no tiene sentido cobrarse entre sí. Tampoco impone Solidity: como solo lo usan empresas, deja elegir lenguajes que esas empresas ya manejan (Go, Java, JavaScript). Mismo objetivo, distinto entorno → distintas soluciones.

PREGUNTA 14 / 16

Deploy once, consume all

Pregunta

Un contrato desplegado lo puede invocar cualquiera del mundo. ¿Es eso una ventaja o un riesgo? ¿Cómo te proteges?

Respuesta razonada

Es las dos cosas a la vez. Ventaja: tu contrato es accesible para todos sin necesidad de servidor, sin permisos, sin caídas. Riesgo: cualquiera puede intentar atacarlo, encontrar bugs, llamarlo de formas inesperadas. La protección no está en "cerrar el acceso" (no se puede), sino en programar bien: validar todos los parámetros, usar control de acceso (onlyOwner), seguir patrones probados (Checks-Effects-Interactions), auditar el código y limitar lo que cada función puede hacer. La filosofía es: si lo despliegas, debe ser robusto frente a un mundo hostil que puede interactuar con él 24/7.

PREGUNTA 15 / 16

Firmas y UX

Pregunta

¿Por qué MetaMask pide firma para cada interacción con un contrato y no funciona como un "login" donde inicias sesión una vez? ¿Qué riesgo aparece si te acostumbras a firmar sin leer?

Respuesta razonada

Porque cada firma autoriza una operación concreta, no una sesión genérica. En un login normal, demuestras quién eres una vez y luego el servidor confía en tu sesión. En blockchain no hay servidor: cada acción que mueve dinero o cambia estado debe estar firmada explícitamente porque solo tú decides. El riesgo es claro: si firmas sin leer, un contrato malicioso puede colarte una autorización que vacía tu wallet (especialmente con "approve" en ERC-20). Por eso MetaMask te muestra cada operación antes de firmar. La fricción es a propósito: es tu seguridad.

PREGUNTA 16 / 16

Datos sensibles en blockchain

Pregunta

Si quisieras guardar una contraseña, un DNI o un dato personal en un Smart Contract, ¿podrías? ¿Por qué es una mala idea?

Respuesta razonada

Técnicamente podrías: cualquier cadena de bytes cabe en una variable de Solidity. Pero es una pésima idea por dos razones. Una: la blockchain es PÚBLICA, todo el mundo puede leerlo, no hay "campo privado". Aunque uses una variable "private" en Solidity, eso solo restringe el acceso desde otros contratos — los datos están en el blockchain a la vista de todos. Dos: es INMUTABLE. Si te equivocas o cambias de opinión, no puedes borrarlo nunca. El criterio práctico: en la cadena solo va lo que quieres que sea público y permanente. Para todo lo demás, hash + almacenamiento fuera de cadena.

Práctica

Buscar información sobre el ERC-3643 en internet y preparar una slide en la que se describen sus principales características.

Práctica

Buscar información sobre el ERC-3643 en internet y preparar una slide en la que se describen sus principales características.

- Estándar de “permissioned tokens” para tokenizar RWA y valores regulados en EVM. Controla quién puede poseer y transferir.
- Usa identidades on-chain y claims verificables para KYC/AML y elegibilidad del inversor antes de emitir o transferir.
- Las reglas de transferencia se validan automáticamente en el smart contract; el emisor mantiene control operativo del activo.
- Compatible con redes públicas y privadas; pensado para cumplimiento en blockchains permissionless.
- Casos de uso: acciones tokenizadas, bonos, fondos, préstamos, inmobiliario y otros RWA sujetos a regulación.
- Beneficios clave: cumplimiento integrado, listas blancas/negras dinámicas vía claims, reducción de fricción legal en mercado secundario.

Fungibilidad

El Concepto de Fungibilidad

El Concepto de Fungibilidad

El Concepto de Fungibilidad

Curiosidades

El Chicago Board of Trade (CBOT) (NYSE: CME) se fundó en 1848, es el mercado de futuros y opciones más antiguo del mundo el cual se ubica en Chicago

La Bolsa de Arroz de Dojima fundada en Osaka, Japón en 1697, es considerada como la primera bolsa de futuros del mundo, era una bolsa de futuros de materias primas especializada en arroz.

Sin embargo, ya en el 2000 A.C. los agricultores del Nilo vendían toda su cosechas antes de que se recogiera, sin saber la cantidad o calidad de sus cultivos.

El Concepto de Fungibilidad

El Concepto de Fungibilidad

Práctica – Fungibilidad

Pedir a los alumnos que identifiquen qué elementos de la lista son fungibles, cuales no y cuales son discutibles:

1. Cartas coleccionables
2. Billetes de 10 €
3. Un cuadro original de Picasso
4. Habitaciones de hotel
5. Bitcoins
6. Monedas antiguas de colección
7. Licencias de software
8. Una casa específica
9. Tokens de un casino
10. Entradas de cine
11. Tu pasaporte
12. Discos de vinilo
13. Camisetas oficiales de un equipo
14. Un contrato firmado con tu nombre
15. Litros de gasolina
16. Un NFT de arte digital único
17. Tickets de transporte público
18. Acciones de una misma empresa
19. Botellas de vino
20. Objetos de videojuegos

Práctica – Fungibilidad

Claramente fungibles (se pueden intercambiar 1:1 sin diferencia)

- Billetes de 10 €
- Litros de gasolina
- Tokens de un casino
- Bitcoins
- Acciones de una misma empresa

Claramente no fungibles (únicos, con identidad propia)

- Un cuadro original de Picasso
- Tu pasaporte
- Un NFT de arte digital único
- Una casa específica
- Un contrato firmado con tu nombre

Práctica – Fungibilidad

Casos ambiguos (dependen del contexto)

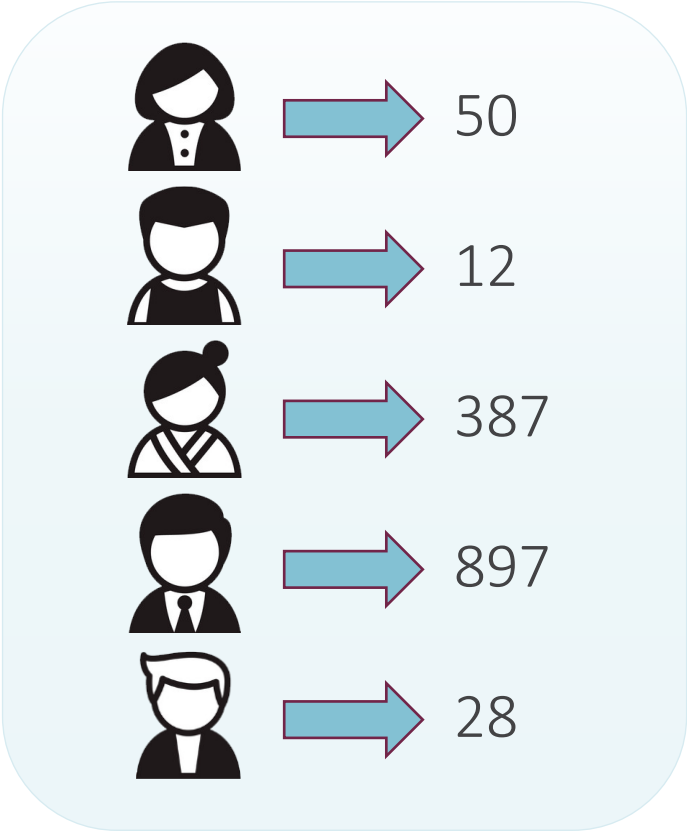
- Botellas de vino (si todas son iguales, fungibles; si cada botella tiene características únicas, no fungible)
- Entradas de cine (si son mismo día/sala/asiento general → fungible; si numeradas → no fungible)
- Camisetas de un equipo (misma talla y edición → fungibles; firmadas por jugadores → no fungibles)
- Cartas coleccionables (una carta común repetida → fungible; una carta rara única → no fungible)
- Monedas antiguas de colección (misma emisión y estado → fungibles; si hay rarezas → no fungibles)
- Objetos de videojuegos (una “poción de salud” → fungible; una espada única con estadísticas diferentes → no fungible)
- Habitaciones de hotel (misma categoría estándar → fungibles; suites únicas → no fungibles)
- Discos de vinilo (reproducciones masivas → fungibles; ediciones limitadas → no fungible)
- Licencias de software (una clave genérica de serie → fungible; una clave personalizada con limitaciones → no fungible)
- Tickets de transporte público (viaje sencillo sin asiento → fungibles; billete con número de asiento → no fungible)

Blockchain

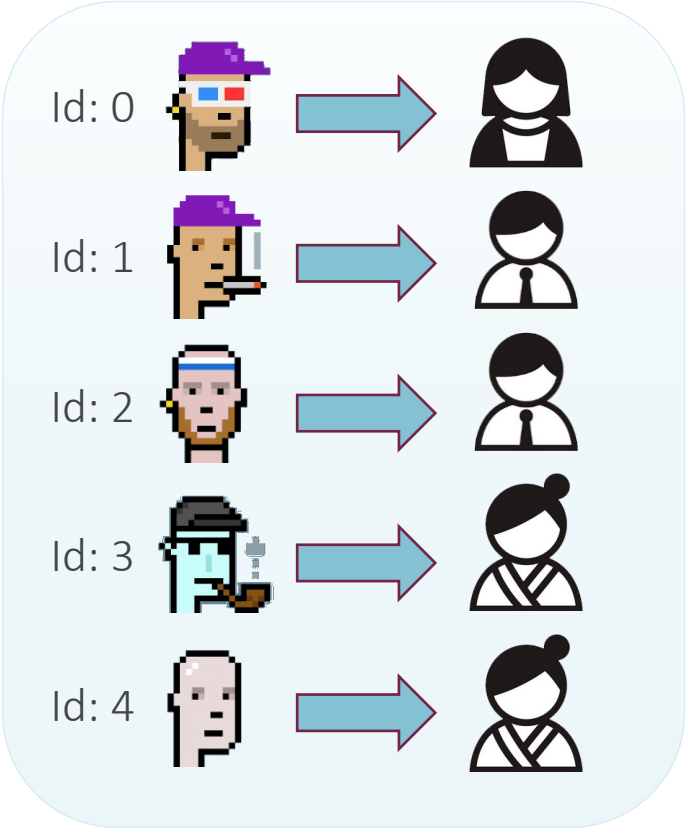
ERC20 ERC721 ERC1155

Smart Contracts – ERC conocidos

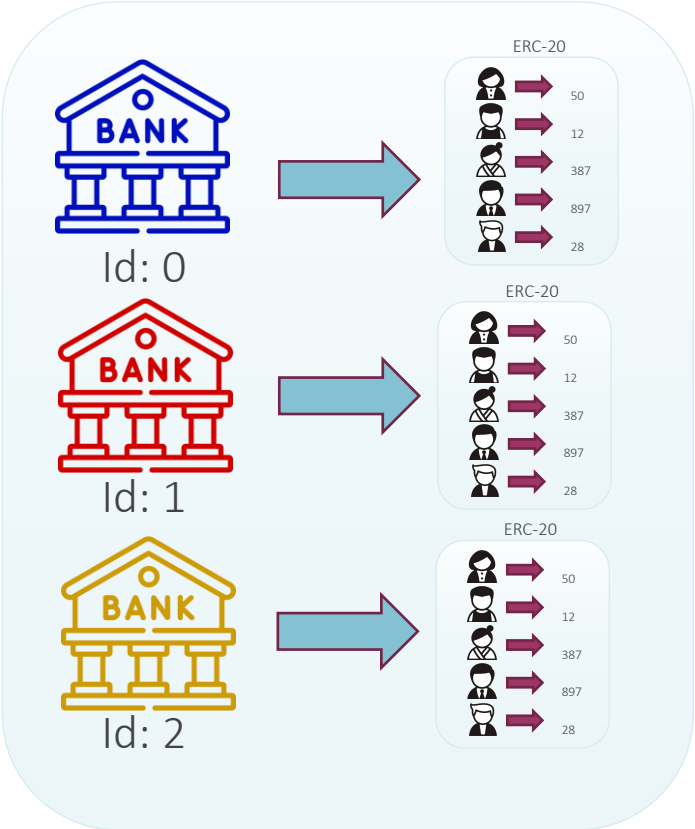
ERC-20



ERC-721



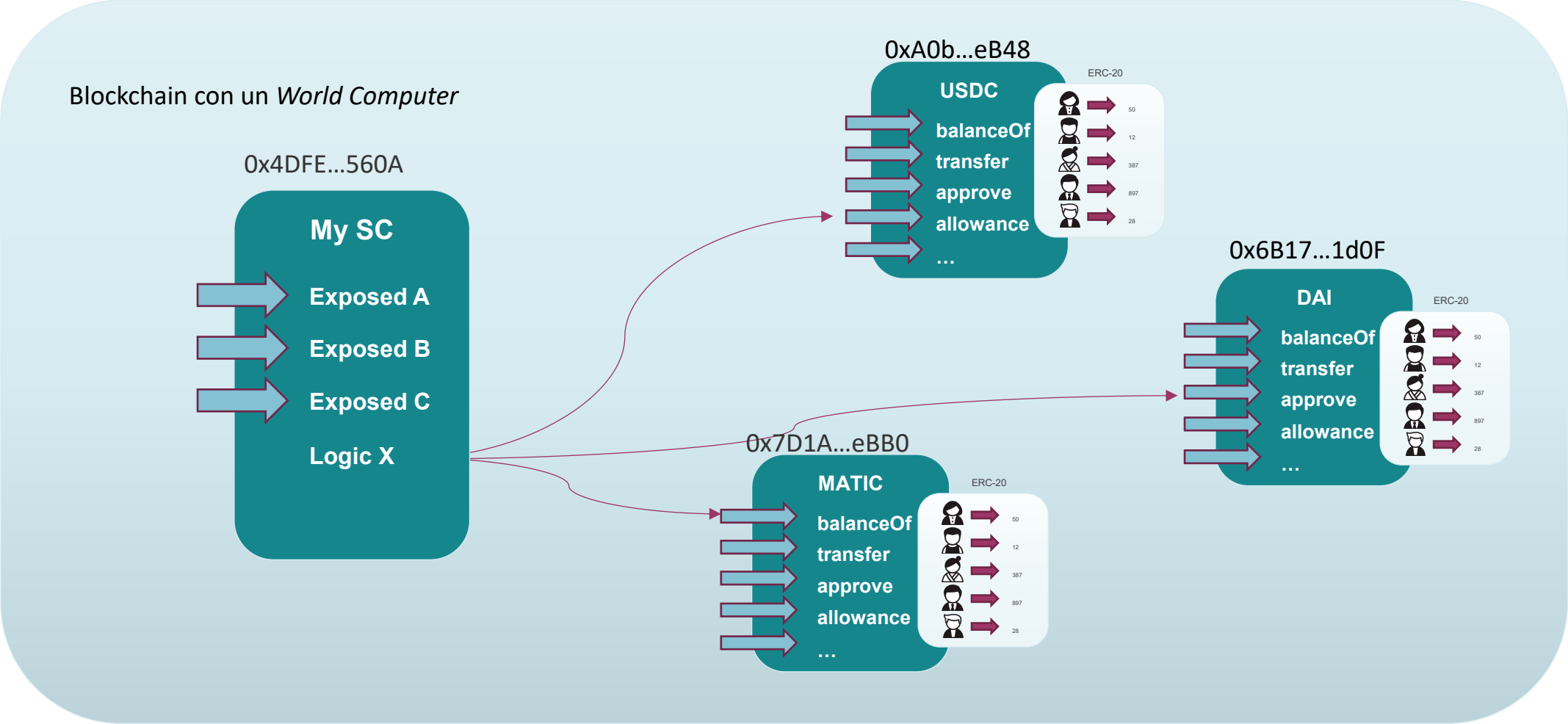
ERC-1155



Smart Contracts – Estandarización

```
function name() public view returns (string)
function symbol() public view returns (string)
function decimals() public view returns (uint8)
function totalSupply() public view returns (uint256)
function balanceOf(address _owner) public view returns (uint256 balance)
function transfer(address _to, uint256 _value) public returns (bool success)
function transferFrom(address _from, address _to, uint256 _value) public returns (bool success)
function approve(address _spender, uint256 _value) public returns (bool success)
function allowance(address _owner, address _spender) public view returns (uint256 remaining)
```

Smart Contracts – Composabilidad



lake of shadows
S
TREASURE HUNT

El contrato misterioso

En Ethereum hay un contrato desplegado en la dirección: 0x06012c8cf97BEaD5deAe237070F9587f8E7A266d

Misión:

- Buscar esa dirección en https://etherscan.io/
- Averiguar qué proyecto famoso es (pista: tiene que ver con gatos)
- ¿Cuántas transacciones ha procesado ese contrato?
- ¿Cuándo se desplegó?
- Encontrar la transacción más cara jamás realizada en ese contrato

Bonus: Ese proyecto causó la primera congestión grave de Ethereum. ¿En qué fecha y cuánto subieron las fees?

Práctica – Contratos Existentes

- ERC20
  - Ir al faucet de Circle: https://faucet.circle.com/
  - Solicitar 10 USDC a nuestro wallet
  - Localizar el address de USDC investigando en Etherscan (https://sepolia.etherscan.io/address/0x1c7D4B196Cb0C7B01d743Fbc6116a902379C7238)
  - Mostrarlo en Metamask
  - Importarlo en Remix. El profesor explicará como hacerlo

Práctica – Crear nuestro propio Token

- Vamos a crear nuestro propio Token ERC20
  - Revisar el Smart Contract entre todos (es una versión mínima de un ERC20)
  - Decidir un nombre entre todos
  - El profesor desplegará el Smart Contract base de ERC20
  - Importar el Token en Metamask
  - El profesor hará un reparto inicial de Tokens a los alumnos
  - Transferir tokens entre alumnos
  - Inspeccionar en sepolia Etherscan las transacciones producidas en la address del Smart Contract

- ¡¡Somos ricos!!